getchar 返回的为什么不是 char：EOF 陷阱

嵌入式 Linux 手记　2026-10-22　公众号文章 · 原链接 mp.weixin.qq.com

先猜结果。这份回显程序，在两种平台上各会出什么事?

```c
#include <stdio.h>

int main(void) {
    char c;
    while ((c = getchar()) != EOF)
        putchar(c);
    return 0;
}
```

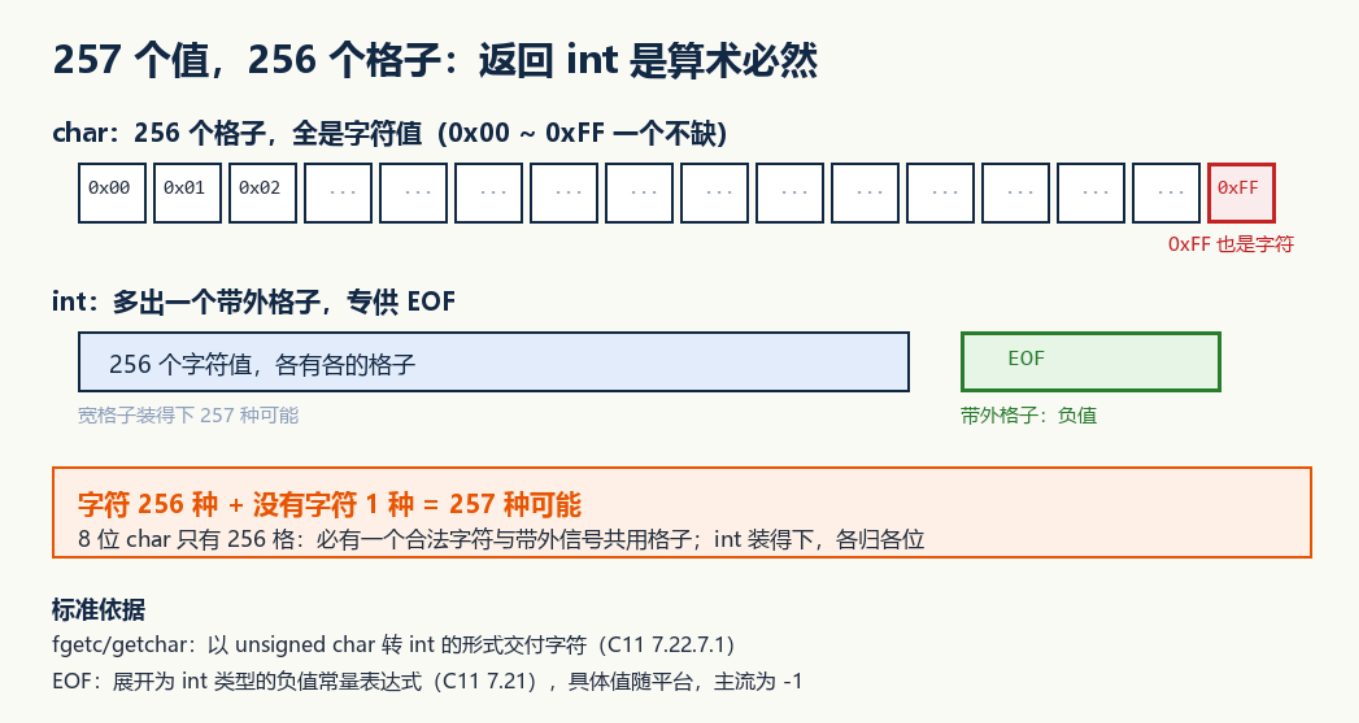
符号 char 的机器上，输入里出现一个字节 `0xFF`，程序当场结束，`0xFF` 后面的内容全部没回显就退出了。无符号 char 的机器上，按 Ctrl+Z（Windows）或 Ctrl+D（Linux）想结束程序，回车之后它毫无反应，永远停不下来。同一份代码，两个方向各崩一种。根子是一个算术事实：getchar 要交出来的值有 257 种，char 只有 256 个格子。

一、257 个值，256 个格子

getchar 的原型一句话就能背错：它读的是字符，返回的偏偏是 int。

```c
int getchar(void);    /* C11 §7.21.7.6 与 fgetc（§7.21.7.1）同族 */
```

标准在 §7.21.7.6 写清了它的行为：从流中取下一个字符，以 unsigned char 转成 int 的形式返回；流尾或出错时返回 EOF。为什么这么绕？因为 getchar 必须用同一条返回通道同时表达两种事："读到了这个字符"和"没有字符可读"。字符有 256 种（一字节），加上"没有字符"这个带外信号，共 257 种可能。8 位的 char 只有 256 个格子，必然有一个合法字符要与带外信号共用格子；int 的量级下限是 32767（§5.2.4.2.1），256 个字符值全放进去还有大把富余，`EOF` 单独占一个带外格子。



图：257 个值与 256 个格子。char 的每个格子都被字符占满，`0xFF` 也是字符；int 多出一个带外格子专供 EOF。

`EOF` 本身在 §7.21 有定义：展开为一个 int 类型的负值常量表达式。具体是几，标准不管，主流平台是 `-1`。

为什么偏要选负值？因为负数区间是唯一的干净空间。fgetc 的交付承诺是"unsigned char 转 int"，正常交付物全部落在 `[0, UCHAR_MAX]`：0 不是空位，那是 NUL 字符；256 以上的数也不是空位，CHAR_BIT 更大的机器上它们都是合法字符值。只有负值区间保证与任何字符值都不重叠，拿它当带外信号，判断才有分辨力。这份精挑细选反过来正是"接进 char 必坏"的算术根源：char 的截断恰好把带外值打回字符值域，分辨力当场清零。

这套"值 + 带外状态挤一条通道"的设计是 K&R 时代的遗产：返回值既携带数据又携带状态，调用方拿一个比较区分两者。它的替代方案现代语言都给了：Rust 的 `Option<u8>` 把数据和状态拆成两个通道，编译器逼你处理"没有"的分支。C 选择省一个参数的代价，把区分的责任压给调用方，于是"格子够不够"变成了第一道生死题。

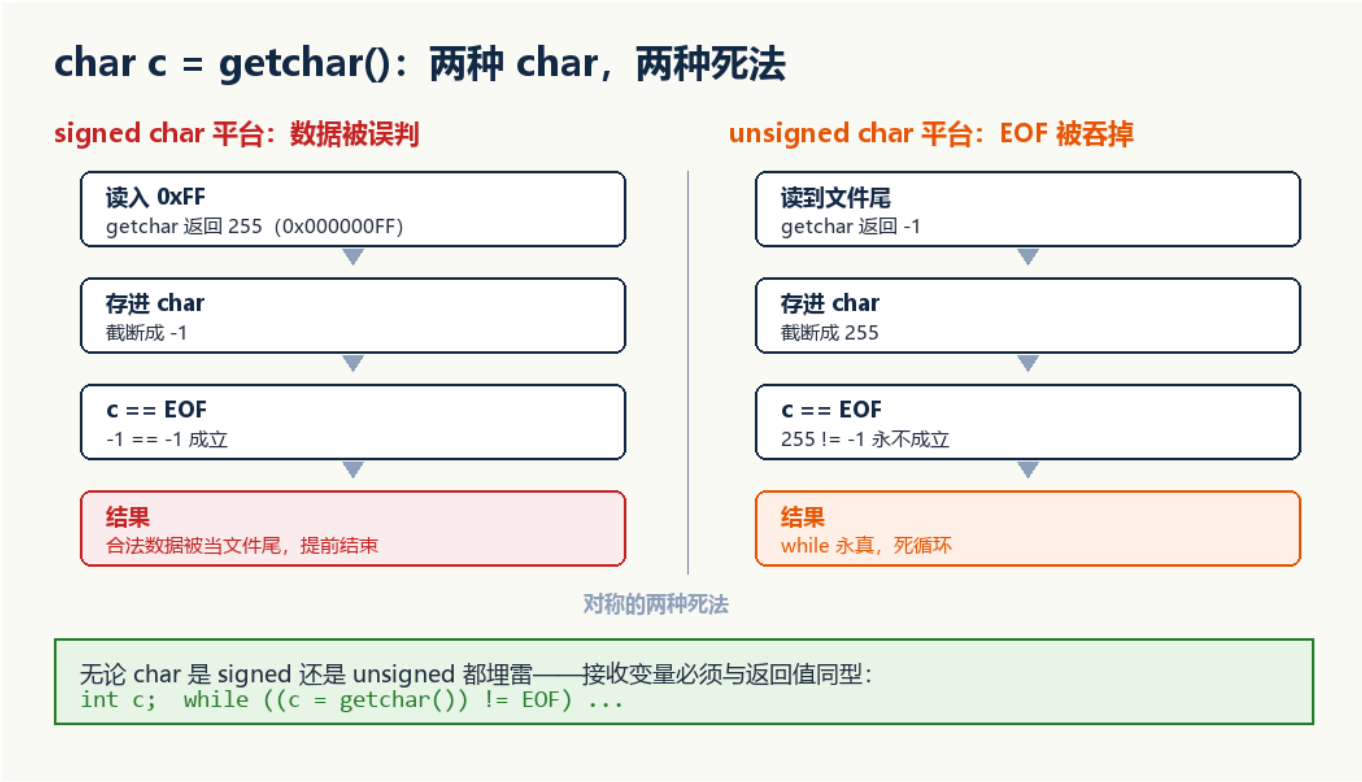
二、两种 char，两种死法

把返回值接进 char，死法取决于平台上 char 的符号性（标准明文交由实现决定，C11 §6.2.5），而且两种都有致命剧本：

```c
char c = getchar();     /* 悲剧的开端 */
if (c == EOF) ...
```

signed char 平台，受害者是数据。读入字节 `0xFF` 时，getchar 如实返回 255（`0x000000FF`）；存进 `char c` 发生截断，`c` 变成 -1；随后 `c == EOF` 的比较里 `c` 整型提升回 int 还是 -1，与 EOF 相等——一个合法的字节被误判成文件尾，`0xFF` 之后的数据全部蒸发。Latin-1 文本里 `0xFF` 是字母 `ÿ`，某些二进制协议里它是常见字段值，谁踩谁知道。

unsigned char 平台，受害者是循环。读到文件尾时 getchar 返回 -1，存进 `char c` 截断成 255；比较时 `c` 提升为 int 是 255，255 永远不等于 -1——EOF 信号被格子吞掉，`while` 条件永真，程序陷入死循环：getchar 每圈都在读尾、putchar 每圈都在往终端刷一个 255 字节，屏幕被 `ÿ` 淹没，进程停不下来。



图：两种 char 两种死法。signed 把 `0xFF` 数据误判成 EOF，unsigned 把 EOF 截成 255 制造死循环，对称的两种翻车。

看清这对称的两种死法：signed 把合法数据错当 EOF（提前结束），unsigned 把 EOF 错当合法数据（永不结束）。无论平台给 char 什么符号性，`char c = getchar()` 都埋着雷，唯一的正确写法是让接收变量与返回值同型：

```c
int c;
while ((c = getchar()) != EOF)
    putchar(c);
```

顺带一句比较语义：`c == EOF` 里发生的是整型提升，C11 §6.3.1.1 规定 int 能完整表示其值域的小整型（char、unsigned char、short）在表达式中一律先升为 int。int 接收时，255 就是 255，-1 就是 -1，257 个格子各归各位，比较才有了分辨力。

三、EOF 之后：真尾还是假错

getchar 返回 EOF 只说明"没有字符可读"，原因有两种：读到流尾，或者读出错。区分它们靠两个状态查询：

```c
int c;
while ((c = getchar()) != EOF) {
    if (c == '\n') lines++;
}
if (ferror(stdin))
    perror("read error");   /* 读出错：磁盘/管道/设备层的失败 */
else
    printf("%d lines\n", lines);   /* 真文件尾 */
```

`feof` 查"是否到达文件尾"，`ferror` 查"是否发生读错误"，两者是流的两个独立标志位。工程上最常见的误用是把 `feof` 当循环条件（`while (!feof(stdin))`），篇 31 会再遇到这个坑，这里只立一条规矩：EOF 是返回值层面的信号，feof/ferror 是状态层面的确认，读到 EOF 后再用后者分辨原因，顺序不能反。写文件出错后还想继续用流，`clearerr` 清掉两个标志位从头再来。

四、同族与延伸：一个返回 int 的家族

getchar 不是孤例，它是 `fgetc` 家族的便捷形态：`fgetc(stdin)` 与 `getchar` 语义完全一致；`getc` 功能相同但允许实现为宏（求值次数有副作用风险的流参数别喂给它）；`getwc` 对宽字符照样返回 `wint_t`，同一个设计模式换了件衣裳。凡族内函数，接收变量一律用 `int`（宽字符用 `wint_t`），这是家族纪律。

回头看这套设计的账本。收益：一条返回通道同时送货和报信，`int c = getchar()` 一行完成取值与判尾，K&R 时代的接口因此干净利落。代价：带外值占用后，合法值域被迫升位到 int，调用方接错类型就从"编译能过"滑向"运行时两种死法"。Rust 的 `Result`、Java 的异常、Go 的多返回值，拆的都是这条通道：数据和状态分家，编译器替你把关。C 没有这些，把关的只有你自己和那一行 `int c`。

写在最后

1. getchar 要表达 257 种可能：256 个字符值加一个带外信号 EOF，8 位 char 装不下，返回 int 是算术必然。
2. EOF 只承诺"负 int"（§7.21），具体值看平台；fgetc 家族以"unsigned char 转 int"的形式交付字符（§7.21.7.6）。
3. 两种 char 两种死法：signed 把 `0xFF` 数据误判成 EOF（提前结束），unsigned 把 EOF 截成 255（永不结束死循环）。接收变量必须 `int c`。
4. EOF 只说"没有字符"，原因二选一：`feof` 真尾、`ferror` 读错；EOF 是返回值信号，feof/ferror 是状态确认，先判返回值再查状态。
5. `fgetc/getc/getchar/getwc` 全族同律：接收类型跟返回值走，`int` 或 `wint_t`，不跟"字符"二字走。

指针深水区还剩最后一块拼图：把指针从内存搬到硬件上。下一篇《0x40000000 不是魔法数：把指针放到寄存器上》，看 Cortex-M 单片机怎么用一个整数地址点亮一颗 LED——虚拟内存的对立面，地址是真的。

合集持续更新，往期与后续篇目都在目录里：C 语言底层实战 · 合集目录

后台回复「交流」，我很乐意分享我的学习计划和一线工作经验，直接来聊。